1) Cel projektu (wersja ankietowa, jedno pytanie otwarte)

Cel główny

Zbudować audytowalny i powtarzalny technicznie pipeline do eksploracyjnej analizy

jakościowej odpowiedzi tekstowych na jedno konkretne pytanie otwarte z ankiety (znamy

question_id i question_text), który:

1. 2. 3. 4. 5. iteracyjnie buduje stateful codebook (stan systemu) dla jednej soczewki analitycznej

(MVP: jedna soczewka),

przypisuje kody do konkretnych miejsc w tekście (dowody) w sposób

weryfikowalny,

przechowuje dane i stan w SQLite, z możliwością łączenia odpowiedzi tej samej

osoby (respondent_id),

automatycznie mierzy minimalny zestaw wskaźników jakości analizy (MVP):

o Evidence grounding: czy dowody/cytaty są dosłowne i poprawnie

zlokalizowane,

o Coverage: co zostało zakodowane, a co pominięte,

o Konwergencja codebooka (code saturation / ITS-proxy): czy tempo

przyrostu unikalnych kodów spada,

umożliwia konsolidację codebooka (merge/split) z zapisem decyzji i

wersjonowaniem.

Założenia MVP

• Analizujemy jedno pytanie otwarte na raz (jedna instancja pipeline'u).

• Nie ma limitu liczby kodów przypisanych do jednego komentarza (multi-label bez

limitu).

• rating (np. NPS 0–10) jest opcjonalnym kontekstem (może być NULL), nie obiektem do kodowania.

• Komentarze są zwykle krótkie (kilkaset znaków), ale pipeline musi obsłużyć rzadkie dłuższe wpisy.

Czego MVP nie robi (na razie)

• Nie buduje tematów (themes) i relacji między tematami.

• Nie robi automatycznej meaning saturation (może być dodane później).

• Nie robi wielosoczewkowości ani koordynacji między soczewkami.

2) Słownik pojęć (definicje wiążące dla projektu)

Survey question (pytanie ankietowe)

Jedno pytanie otwarte, które analizujemy w danym przebiegu:

• question_id: identyfikator pytania,• question_text: treść pytania (kontekst semantyczny).

Response / Comment (odpowiedź / komentarz)

Pojedyncza odpowiedź tekstowa respondenta na analizowane pytanie.

Minimalne pola:

• comment_id (unikalne),

• respondent_id (może się powtarzać między rekordami),

• question_id,

• comment_text (tekst odpowiedzi),

• rating (opcjonalnie; np. 0–10; może być brak).

Canonical text (tekst kanoniczny)

Jedyna, ustalona wersja tekstu komentarza, na której liczone są offsety (pozycje znaków).

• W MVP canonicalizacja jest minimalna: stabilny whitespace, brak parafraz, brak automatycznej interpunkcji.

• Offsety zawsze odnoszą się do canonical text.

Span (zakres tekstu)

Ciągły fragment canonical text określony przez:

• start_char i end_char (w obrębie canonical text danego komentarza).

Span to adres w tekście, który umożliwia weryfikację dowodu.

Segment (jednostka robocza w bazie)

Predefiniowany fragment komentarza zapisany w SQLite, z własnym segment_id.

• W standardowym przypadku: 1 komentarz = 1 segment.

• Gdy komentarz jest długi: komentarz dzielimy na kilka segmentów.

Segment ma:

• comment_id,

• start_char, end_char (span w canonical text komentarza),

• text (substring canonical text).

Segment jest jednostką do:

• coverage,

• przypisań kodów,

• budowy okien do LLM,

• audytu i human review.

Window / Chunk (okno kontekstu do LLM)

Tekst wejściowy dla LLM zbudowany przez sklejenie jednego lub kilku segmentów.• Dla krótkich komentarzy: okno = 1 komentarz (1 segment).

• Dla długich komentarzy: okno = kilka segmentów (z overlap), żeby zachować

kontekst.

Evidence quote (dowód / cytat)

Dosłowny fragment tekstu (substring canonical text) użyty jako uzasadnienie przypisania

kodu.

W MVP cytat musi być:

• dosłowny (da się sprawdzić substring match),

• osadzony w konkretnym segment_id i/lub w spanach.

Code (kod)

Element codebooka: etykieta + definicja + przykłady (anchor quotes).

Kod ma stabilne code_id i należy do jednej soczewki.

Code assignment (przypisanie kodu)

Rekord mówiący, że kod code_id odnosi się do konkretnego segmentu komentarza

(segment_id) z konkretnym dowodem (cytat/span).

Codebook (codebook jako stan systemu)

Zbiór kodów i ich bieżących definicji oraz przykładów, który jest aktualizowany iteracyjnie.

Codebook jest stateful i wersjonowany.

Lens (soczewka analityczna)

Zestaw instrukcji “jak kodować” (np. produktowo-użytecznościowa).

MVP: jedna soczewka.

Grounding (evidence grounding)

Własność przypisania kodu, że:

• cytat/dowód istnieje dosłownie w danych,

• jest poprawnie zlokalizowany (segment/span),

• nie jest parafrazą udającą cytat.

Coverage

Stopień, w jakim odpowiedzi/segmenty zostały objęte kodowaniem (oraz identyfikacja

braków).

Konwergencja codebooka (ITS-proxy / code saturation w MVP)Sygnał, że przyrost unikalnych kodów spowalnia względem liczby wszystkich

wygenerowanych kandydatów, co sugeruje stabilizację codebooka w danej soczewce.

3) Etap 0 — Pre-processing dla ankiet (specyfikacja

architektury)

3.1 Cel etapu 0

Przekształcić surowe odpowiedzi ankietowe do postaci:

• jednoznacznej (canonical text + offsety),

• mierzalnej (segmenty do coverage),

• gotowej do kodowania (okna do LLM),

• zapisanej w SQLite w sposób odporny na zmiany w dalszych etapach.

3.2 Wejścia (input contract)

Zbiór odpowiedzi na jedno pytanie, gdzie każdy rekord ma:

• comment_id

• respondent_id

• question_id

• question_text

• comment_text

• rating (opcjonalnie; może być NULL)

Założenie: analizujemy tylko rekordy z tym samym question_id.

3.3 Wyjścia (output contract, SQLite)

Minimalne tabele:

questions

• question_id (PK)

• question_text

• question_version (opcjonalnie; jeśli pytania się zmieniają w czasie)

responses

• comment_id (PK)

• respondent_id

• question_id (FK)

• rating (NULL allowed)

• raw_text (oryginał)

• canonical_text (po canonicalizacji)

• canonicalization_version• text_length_chars

• flags (np. empty/very_short/noise)

segments

• segment_id (PK)

• comment_id (FK)

• seq_idx (kolejność segmentu w obrębie komentarza)

• start_char, end_char (w canonical_text komentarza)

• text (substring)

• flags (np. long_split, low_quality)

windows

• window_id (PK)

• comment_id (FK) lub (opcjonalnie) group_id jeśli kiedyś batchujesz wiele

komentarzy

• segment_ids (lub tabela łącząca window_segments)

• window_text

• window_char_len

• overlap_info (jeśli dotyczy)

preprocess_report

• agregaty dla pytania: liczba odpowiedzi, odsetek pustych, rozkład długości, liczba

podzielonych komentarzy, itp.

3.4 Reguły etapu 0 (krok po kroku)

Krok 0A — Walidacja i filtrowanie wejścia

1. 2. 3. Usuń rekordy bez comment_id lub bez comment_text (techniczne braki).

Zidentyfikuj komentarze puste / whitespace-only:

o oznacz flagą empty_text

o nie twórz segmentów i okien dla pustych (bo nie ma co kodować)

Zidentyfikuj komentarze ekstremalnie krótkie (np. 1–2 znaki):

o oznacz flagą too_short

o nadal zapisuj w responses, ale segmentacja może być trywialna.

Uwaga: nie filtrujesz “wszystko ok/super” na tym etapie. To jest decyzja analityczna później.

Krok 0B — Canonicalizacja tekstu komentarza

Dla każdego komentarza:

1. Zachowaj raw_text.

2. Zbuduj canonical_text przez:

o normalizację whitespace: wielokrotne spacje → jedna, ujednolicenie \r\n →

\n,

o przycięcie spacji na początku/końcu,o brak automatycznej korekty pisowni i brak dopisywania interpunkcji (MVP).

3. Zapisz canonical_text + canonicalization_version.

Dlaczego minimalna canonicalizacja: utrzymuje stabilne offsety i minimalizuje ryzyko

“przesunięć” spanów.

Krok 0C — Segmentacja (małe segmenty w DB)

Domyślna reguła:

• jeśli len(canonical_text) ≤ LONG_COMMENT_THRESHOLD, to:

o twórz 1 segment obejmujący cały komentarz (start_char=0, end_char=len).

Reguła dla rzadkich długich komentarzy:

• jeśli len(canonical_text) > LONG_COMMENT_THRESHOLD, to dzielisz na segmenty.

Parametry startowe (jawne w configu):

• LONG_COMMENT_THRESHOLD: np. 1200–2000 znaków (ustalasz; zależy od języka i praktyki)

• TARGET_SEG_CHARS: np. 600–900 znaków

• MAX_SEG_CHARS: np. 1200–1500 znaków

• MIN_SEG_CHARS: np. 200–300 znaków

Reguły dzielenia:

1. Preferowane granice cięcia:

o znaki nowej linii,

o interpunkcja kończąca zdanie (jeśli występuje),

o średniki, dwukropki,

o w ostateczności: twardy limit znaków.

2. Po podziale:

o scal segmenty krótsze niż MIN_SEG_CHARS z sąsiadem (jeśli nie przekraczasz MAX_SEG_CHARS po scaleniu).

Każdy segment dostaje seq_idx rosnące od 0.

Krok 0D — Budowa okien kontekstu (windows) do LLM

Ponieważ komentarze są zwykle krótkie:

• dla komentarzy 1-segmentowych: window = segment.

• dla komentarzy wielosegmentowych: budujesz sliding window po segmentach.

Parametry startowe:

• TARGET_WINDOW_CHARS: np. 1200–2500 znaków

• WINDOW_OVERLAP_SEGMENTS: np. 1 segment overlapReguły:

1. Sklej segmenty w okno, aż osiągniesz TARGET_WINDOW_CHARS (lub wyczerpiesz

2. 3. segmenty).

Dodaj separatory między segmentami (np. \n---\n).

Kolejne okno zaczynasz z przesunięciem uwzględniającym overlap.

Krok 0E — Kontekst pytania i ratingu (jak to trafia do LLM)

Na etapie 0 nie mieszasz pytania z tekstem komentarza w canonical_text. Pytanie i rating są metadanymi.

W późniejszym Etapie 1 okno do LLM będzie budowane jako:

• question_text (kontekst),

• opcjonalnie rating,

• window_text (materiał do kodowania).

To gwarantuje, że cytaty/offsety zawsze odnoszą się tylko do komentarza.

Krok 0F — Raport kontrolny preprocessingu (obowiązkowy)

Dla question_id zapisujesz agregaty:

• liczba wszystkich rekordów wejściowych,

• liczba pustych (empty_text),

• rozkład długości canonical_text (min/median/p95/max),

• liczba komentarzy przekraczających LONG_COMMENT_THRESHOLD,

• liczba segmentów i okien łącznie,

• ile komentarzy ma rating vs brak ratingu.

To jest sanity-check dla dalszych etapów i do tuningu progów.

3.5 Walidacje (co musi przejść, żeby iść dalej)

• Każdy segment.text musi być dokładnym substringiem

responses.canonical_text w zakresie start_char:end_char.

• Segmenty w obrębie komentarza:

o nie mogą się nakładać,

o muszą zachować kolejność,

o ich konkatenacja (z ewentualnymi separatorami usuniętymi) ma pokrywać cały

canonical_text (dla komentarzy wielosegmentowych).

• Każdy window musi mapować się do listy segmentów (traceability).

3.6 Logowanie/audyt

W event log (w SQLite lub osobnej tabeli) zapisujesz:

• wersje progów segmentacji,

• canonicalization_version,• liczbę split/merge,

• identyfikatory komentarzy, które zostały podzielone (do ręcznego przeglądu).

3.7 Failure modes (co może pójść źle i jak to flagować)

• Komentarze z “śmieciowym” tekstem (same znaki specjalne) → flag noise.

• Bardzo długie komentarze bez interpunkcji → flag hard_split_used.

• Dane wejściowe zawierają duplikaty comment_id → blokada lub deduplikacja z

logiem.

To domyka Etap 0 dla ankiet. Następny krok (Etap 1) będzie już opisywał dokładnie: jak budujemy wejście do LLM (question_text + rating + window_text), jak LLM zwraca kandydatów kodów i jak wymagamy cytatów/spanów, żeby grounding był mechanicznie

walidowalny.

Etap 1 — Initial coding (ankiety, jedno pytanie otwarte)

Poniżej specyfikacja w tym samym rygorze: cel → wejścia → wyjścia → reguły → walidacje → log/audyt → failure modes. Etap 1 nie aktualizuje jeszcze codebooka (to robi Etap 2: dedupe/matching). Etap 1 produkuje kandydatów.

1) Cel etapu 1

Dla każdego komentarza (lub okna komentarza, jeśli komentarz podzielony na segmenty) wygenerować:

1. 2. 3. kandydatów kodów (labels) zgodnie z wybraną soczewką,

przypisania tych kandydatów do konkretnych miejsc w tekście (segment_id + cytat jako dowód),

wstępną definicję kodu (1–2 zdania) dla nowych kandydatów, z założeniem, że definicja może się później zmieniać.

Ważne: Etap 1 nie może tworzyć „wniosków bez dowodu”. Każde przypisanie ma mieć cytat.

2) Wejścia (z SQLite)

Kontekst badania (stały dla całej instancji pipeline’u)

• question_id

• question_text• lens_id + lens_spec (opis soczewki: jak kodować)

Dane do kodowania

• responses (dla danego question_id): comment_id, respondent_id, rating (opcjonalnie), canonical_text

• segments: segmenty dla komentarzy

• windows: okna zbudowane z segmentów

Stan codebooka (read-only w Etapie 1)

Etap 1 może czytać aktualny codebook danej soczewki (żeby ograniczyć dryf), ale nie zapisuje zmian do codebooka. Zmiany idą w Etapie 2.

3) Wyjścia (do SQLite)

Tabela candidate_codes

Rekordy kandydatów kodów wykrytych w danym przebiegu:

• candidate_code_id

• run_id

• lens_id

• label (proponowana nazwa kodu)

• definition_stub (1–2 zdania)

• is_existing_hint (bool/nullable) — czy LLM sugeruje, że to pasuje do istniejącego kodu (hint, nie decyzja)

Tabela candidate_assignments

Przypisania kandydatów do tekstu:

• candidate_assignment_id

• run_id

• lens_id

• comment_id

• window_id

• segment_id (segment, w którym znajduje się dowód)

• candidate_code_id

• quote_exact (dosłowny substring)

• quote_start_char, quote_end_char (offsety w canonical_text komentarza; jeśli na tym etapie nie liczysz, mogą być NULL i wyliczysz w walidacji)

• context_note (opcjonalnie 1 zdanie: „co tu się dzieje”)

• polarity (opcjonalnie: positive/negative/neutral/mixed; jeśli chcesz od razu)

Tabela stage1_reportAgregaty:

• liczba przeprocesowanych komentarzy/okien

• liczba kandydatów kodów

• liczba przypisań

• średnia liczba przypisań na komentarz

• odsetek komentarzy bez żadnego przypisania (do coverage)

4) Reguły i parametry (jawne)

Parametry uruchomienia (konfiguracja etapu)

• RUN_MODE: all_comments (MVP)

• BATCHING: czy kodujemy per komentarz, czy per paczka komentarzy:

o rekomendacja MVP: per window, ale okna mogą obejmować jeden

komentarz; paczkowanie zostaw jako opcję dopiero jeśli koszty będą

problemem

• MAX_COMMENTS_PER_PROMPT (jeśli paczkujesz): np. 3–8

• INCLUDE_RATING_AS_CONTEXT: true/false (MVP: true, ale rating może być

NULL)

• RETURN_SCHEMA_STRICT: true (LLM musi zwrócić JSON zgodny ze schematem)

Zasady kodowania

1. 2. 3. 4. 5. 6. Kodujesz tylko tekst odpowiedzi respondenta (window_text).

question_text i rating są kontekstem, nie materiałem do cytowania.

Każdy kandydat kodu musi mieć:

o label

o definicję (stub)

o co najmniej jedno przypisanie z cytatem (quote_exact)

Cytat musi być dosłowny i pochodzić z segmentu (segment_id).

Brak limitu liczby kodów na komentarz (Twoja decyzja).

Jeżeli komentarz jest ogólnikowy („ok”, „super”, „wszystko w porządku”) to:

o nadal może dostać kod typu „ogólna pozytywna ocena / brak szczegółów”

o ale cytat nadal jest wymagany.

5) Budowa wejścia do LLM (co dokładnie wysyłamy)

Dla każdego window (zwykle 1 komentarz):

• Header:

o question_id, question_text

o comment_id, respondent_id

o rating (jeśli istnieje; jeśli nie, explicit NULL)

o lens_spec (instrukcja soczewki)• Data:

o lista segmentów wchodzących w window, w formacie:

▪ segment_id

▪ segment_text

o (ważne) segmenty muszą być rozdzielone separatorami, ale segment_id musi

być widoczny dla LLM.

LLM powinien pracować na segmentach, żeby zwracać przypisania do segment_id, a nie

„gdzieś w tekście”.

6) Wymagany format odpowiedzi LLM (kontrakt)

LLM zwraca strukturę:

• candidate_codes[]

o label

o definition_stub

• assignments[]

o label (nazwa kodu jak wyżej; Etap 2 zamieni to na code_id)

o segment_id

o quote_exact

o opcjonalnie: context_note, polarity

Kontrakt musi wymuszać, że:

• assignments odwołują się do istniejących segment_id w oknie,

• quote_exact jest substring segment_text.

7) Walidacje w Etapie 1 (minimalne, żeby nie śmiecić

bazy)

Etap 1 robi walidację „syntaktyczną” i „mechaniczną”:

1. 2. 3. 4. Schema validation: JSON parsowalny i zgodny ze schematem.

Segment ID validation: każde assignment wskazuje segment_id obecny w window.

Exact quote check: quote_exact jest dosłownym substringiem segments.text dla

wskazanego segment_id.

(opcjonalnie) Quote length rule: cytat nie może być krótszy niż np. 20–40 znaków,

chyba że komentarz jest bardzo krótki (wtedy dopuszczasz wyjątek).

Jeśli walidacja nie przejdzie:

• rekordy z tego window trafiają do stage1_errors z opisem błędu,

• a window może zostać ponowione z bardziej restrykcyjnym promptem (w MVP: max

1 retry).8) Logowanie i audyt

Dla każdego runu zapisujesz:

• run_id

• question_id, lens_id

• model_name, parametry

• wersję promptu (prompt_hash)

• liczbę okien/komentarzy

• liczbę błędów walidacji + ich typy

• opcjonalnie: surowe odpowiedzi LLM (albo ich hash) dla odtwarzalności

9) Failure modes i jak je obsłużyć

1. LLM zwraca parafrazę zamiast cytatu

→ fail na exact quote check, retry z instrukcją „cytat musi być dosłowny substring".

2. LLM miesza segment_id lub wymyśla segment_id

→ fail na segment validation, retry z krótszym oknem lub bardziej czytelną listą segmentów.

3. Komentarze bardzo krótkie ("ok")

→ dopuszczasz krótkie cytaty, ale nadal wymagane; kod typu „ogólnik".

4. Komentarz zawiera kilka tematów

→ bez limitu: wiele assignments; w raporcie coverage/obciążenia to wyjdzie.

5. Skala 26k: koszty i czas

→ w MVP trzymasz per-window, ale równoleglisz przetwarzanie (batch w sensie równoległych requestów), nie "paczki komentarzy w jednym prompt" na start.

Jedna decyzja przed przejściem do Etapu 2

W Etapie 1 sugerowałem per-window (zwykle 1 komentarz). Alternatywa to paczkowanie 3–8 komentarzy w jednym prompt.

Pytanie: w MVP chcesz kodować pojedyncze komentarze (per window), czy od razu

paczkować kilka komentarzy w jednym prompt (dla kosztu/przepustowości)?